OCS INTELLIGENCE · SPRINT 4 · PLAYBOOK

Rig routing

Putting the OCS Intelligence rigs on a private network behind the filtered school Wi-Fi, and routing them through an EC2 WireGuard relay.

STATE **in progress** | DATE 2026-10-16 | PROJECT PROMX-OCS | SOURCE evidence/sprint4/2026-10-16-rig-routing.md

The room's wall Ethernet jacks don't work, so the only way out is the school Wi-Fi, which filters traffic and blocks unusual ports. We gave the rigs their own private network. A Mac mini joins the school Wi-Fi and shares that connection over Ethernet to a GL.iNet Flint 2 router. The router runs the OCS LAN: both rigs are wired into it, and it broadcasts the 5 GHz `OCS_Intel` Wi-Fi. That part is live and measured: each rig links at 1 Gb/s, rig to rig carries 934 Mb/s, and the internet gets ~84 Mb/s shared. The second half, a WireGuard tunnel from the Flint to the EC2 relay so all OCS traffic leaves from EC2, is planned and continues in Sprint 5.

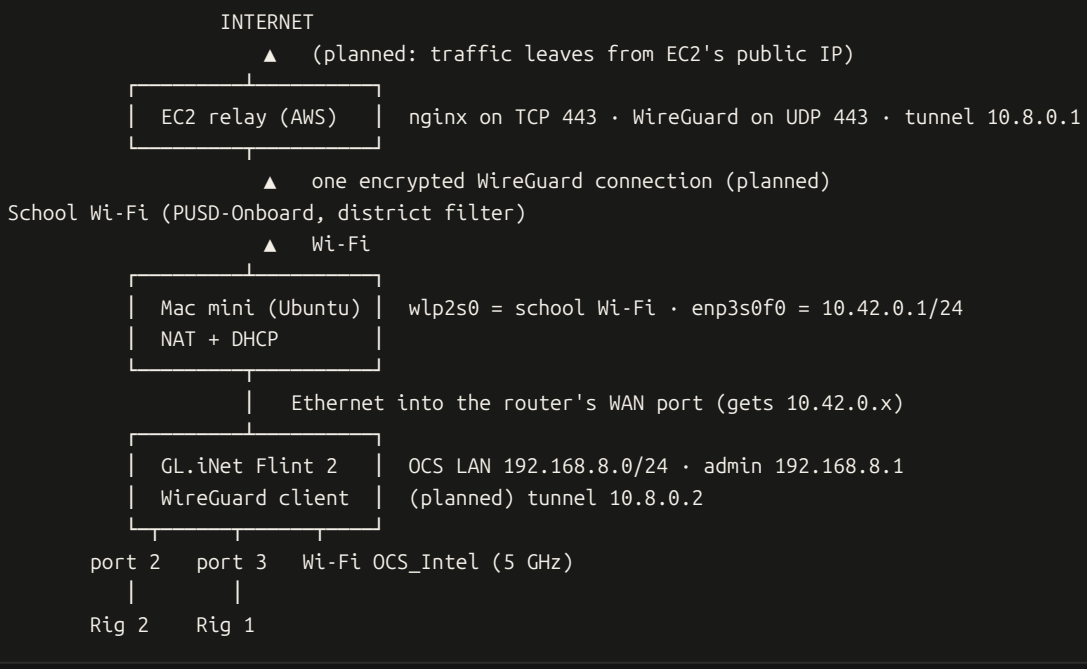
This playbook was first written by the team as a PDF on 2026-10-16. It was converted to the standard format on 2026-10-23 and corrected where the setup has changed: the Wi-Fi is `OCS_Intel` (it was `OCS-IoT`), and Rig 2 is wired (it was on a USB Wi-Fi dongle).

At a glance

Goal	The rigs on a private LAN we control, with all of their internet traffic leaving through EC2
Outcome	**Live:** Mac mini uplink, Flint 2 router, both rigs wired, `OCS_Intel` Wi-Fi. **Planned:** the WireGuard tunnel to EC2 (Steps 6–10)
Where	Mac mini (Ubuntu), GL.iNet Flint 2 (GL-MT6000), both rigs, the EC2 relay
Why	No decision record yet. Defining the "closed-loop" network is a Sprint 5 item

PIECE	STATUS
Mac mini joins the school Wi-Fi on its built-in card	Live. Proprietary `wl` driver, profile `PUSD open test`
Mac mini shares its connection over Ethernet	Live. NetworkManager shared mode on `enp3s0f0`, `10.42.0.1/24`
Flint 2 gets its WAN address from the Mac mini	Live. Tested first with a Brume 2 (got `10.42.0.185`), then the Flint 2
Flint 2 runs the OCS LAN and `OCS_Intel` Wi-Fi	Live. 5 GHz only; 2.4 GHz is off (2026-10-23)
Rig 2 wired to Flint LAN port 2, Rig 1 to port 3	Live (ports reported 2026-10-23). Port 4 is for orchestrator testing; ports 5 and 6 belong to other groups in the class

PIECE	STATUS
Reboot test of the Mac mini with its USB dongle unplugged	Pending. Must pass before anything depends on it
UDP 443 reachability from the Mac mini to EC2	Pending. Decides the tunnel's port
WireGuard server on EC2, Flint 2 as client	Planned (Steps 6–9)
EC2 reaches the rigs through the tunnel	Planned (Step 10)



Before

- The room's wall Ethernet jacks don't work. The only uplink is the school Wi-Fi, `PUSD-Onboard`.
- The district filters outbound traffic and blocks unusual ports, so a tunnel has to use a port that gets through.
- Consumer routers can't join the school network in a way we control, so a Linux machine (the Mac mini) joins it and hands the connection to our own router.

> **IMPORTANT**
>
> The OCS team has permission for this network. If you build this somewhere else, get written approval from the network owner first. Tunneling past a filter without approval breaks acceptable-use rules and can get the whole uplink blocked.

Why the WireGuard server goes on EC2, not the router. Encryption only protects traffic up to where the tunnel ends. If the router were the server, traffic would be decrypted inside the room and cross the school filter in the clear. A server also waits for inbound connections, and the school network blocks those. A client connects outward, which gets through. EC2 has a public address and an open port, so EC2 is the server and the router is the client.

Why the Flint 2 and not the Brume 2. The Flint 2 (GL-MT6000) has Wi-Fi 6 and 1 WAN + 5 LAN ports, so it does wired LAN, Wi-Fi, and the WireGuard client in one box. The Brume 2 (GL-MT2500A) has no Wi-Fi radio and 1 WAN + 1 LAN. It's kept as a spare.

Hardware and addresses

ITEM	DETAILS
Mac mini	Ubuntu with NetworkManager; the school-facing uplink
Its built-in Wi-Fi	Broadcom BCM4360, PCI ID `14e4:43a0`, interface `wlp2s0`. Needs the proprietary `wl` driver
Its Ethernet	Broadcom NetXtreme Gigabit, `enp3s0f0`
USB Wi-Fi dongle (fallback)	Ralink RT5370, USB ID `148f:5370`, in-kernel `rt2800usb` driver. Supports OWE
Router	GL.iNet Flint 2 (GL-MT6000). Brume 2 kept as a spare
Relay	Ubuntu EC2 instance with an Elastic IP. nginx already runs on TCP 443

NETWORK	SUBNET	WHO HANDS OUT ADDRESSES	FIXED ADDRESSES
School Wi-Fi	Whatever the district gives	District DHCP	None
Mac mini → router link	`10.42.0.0/24`	Mac mini (NetworkManager shared mode)	Mac mini = `10.42.0.1`
OCS LAN (wired + `OCS_Intel`)	`192.168.8.0/24`	Flint 2	Flint = `192.168.8.1`; Rig 1 = `192.168.8.112`, Rig 2 = `192.168.8.213` (seen 2026-10-22/23)
WireGuard tunnel (planned)	`10.8.0.0/24`	Static, in the configs	EC2 = `10.8.0.1`, Flint = `10.8.0.2`

Every subnet must differ from the others, or routing breaks. `10.8.0.0/24` is only a common WireGuard convention; any private range that doesn't overlap the others works.

Before you start: SSH with sudo on the EC2 relay and AWS console access to its security group; the relay's Elastic IP (`dig +short ai.opencodingsociety.com`); a keyboard and monitor (or SSH) on the Mac mini, with sudo; Ethernet cables (Mac mini to the Flint's WAN port, one per wired device). Steps 1–5 take about 2 hours, Steps 6–9 about 1 hour.

What we did

Step 1: Get the Mac mini's built-in Wi-Fi working

Ubuntu doesn't ship a working driver for the BCM4360, so the card is invisible until you install one.

1. Find the card by Broadcom's vendor ID (a plain `grep network` can miss it):

```
lspci -nn | grep -i 14e4      # expect a line ending in [14e4:43a0] -> BCM4360
```

With a USB dongle too, compare `lsusb` with it unplugged and plugged in. `0a5c:4500` is the Bluetooth controller, not Wi-Fi.

2. Install the driver and reboot:

```
sudo apt update
sudo apt install -y dkms linux-headers-$(uname -r) broadcom-sta-dkms
sudo reboot
```

3. Check it:

```
dkms status                          # broadcom-sta ... installed
lsmod | grep -E "wl|b43|bcma|ssb"    # wl loaded; b43/bcma/ssb NOT loaded
nmcli device                         # wlp2s0 listed as wifi
nmcli device wifi list ifname wlp2s0 # school SSIDs show up
```

> **WARNING**
>
> With Secure Boot on, the DKMS module must be enrolled (MOK) or it won't load. A kernel update can also break the rebuild. Re-run the checks above after every kernel update.

Step 2: Join the school Wi-Fi from the Mac mini

`PUSD-Onboard` uses OWE (encrypted open Wi-Fi) in transition mode, so it appears as a pair: an OWE side and a plain open side. The `wl` driver can't do OWE, so the built-in card joins the open side, pinned to one access point.

1. Pick the access point. Choose the strongest entry **not** marked OWE and copy its BSSID:

```
nmcli -f SSID,BSSID,SECURITY,CHAN,SIGNAL device wifi list ifname wlp2s0 | grep -i onboard
```

2. Create the profile:

```
nmcli connection add type wifi ifname wlp2s0 con-name "PUSD open test" \
    ssid PUSD-Onboard 802-11-wireless.bssid <BSSID_FROM_ABOVE> \
    connection.autoconnect yes
nmcli connection up "PUSD open test"
```

3. Check it: `ip route` must show the default route on `dev wlp2s0`, and `curl -I https://example.com` must return headers. `ping 8.8.8.8` fails here, and that's expected: the school network appears to drop ICMP. Test with `curl`.

What failed, so nobody repeats it:

ATTEMPT	RESULT
Clone the dongle's OWE profile onto `wlp2s0`	Stuck configuring; association took too long. Deleted
Point the original OWE profile at `wlp2s0`	Network not found (`wl` can't do OWE)
OWE profile pinned to one BSSID	Network not found

ATTEMPT	RESULT
New open profile pinned to one BSSID	**Worked**

At a different school: if the card supports OWE (most mac80211 drivers, like the RT5370), use `wifi-sec.key-mgmt owe` for encryption. With WPA2-Enterprise (PEAP/MSCHAPv2), add the 802-1x settings with school credentials instead.

Step 3: Share the Mac mini's connection over Ethernet

NetworkManager's shared mode turns on IP forwarding, NAT, and a small DHCP server on the Ethernet port in one command.

1. Plug a cable from the Mac mini into the router's **WAN** port.
2. Create and start the shared profile:

```
nmcli connection add type ethernet ifname enp3s0f0 con-name "brume-link" \
    ipv4.method shared connection.autoconnect yes
nmcli connection up "brume-link"
```

3. Check it:

```
nmcli device                                    # enp3s0f0 connected, brume-link
ip addr show enp3s0f0                           # inet 10.42.0.1/24
sysctl net.ipv4.ip_forward                      # = 1
cat /var/lib/NetworkManager/dnsmasq-enp3s0f0.leases   # the router's WAN lease appears
```

The profile is called `brume-link` because it was first tested with the Brume 2. The name doesn't matter; it works the same with the Flint 2.

Step 4: Set up the Flint 2

1. Connect a computer to the Flint (a cable to a LAN port, or the default Wi-Fi on its label).
2. Open `http://192.168.8.1`. Set a strong admin password and write it down somewhere private: anything on the OCS LAN can reach this page.
3. **Internet:** it should already show Ethernet connected by DHCP with a `10.42.0.x` address. If so, leave it, and don't change the network mode.
4. **Wireless:** name the network `OCS_Intel`, use WPA2/WPA3, and set a strong password. The team runs it on 5 GHz only, with 2.4 GHz off.
5. From that computer, `curl -I https://example.com`. Headers back means the whole chain works: school Wi-Fi → Mac mini → Flint.

Step 5: Connect the rigs

Both rigs are wired (Rig 2 moved off its temporary USB Wi-Fi dongle by 2026-10-20).

1. Run an Ethernet cable from each rig to a Flint LAN port: **Rig 2 to port 2, Rig 1 to port 3.** Port 4 is for orchestrator testing. Ports 5 and 6 are used by other groups in the class.

2. In the Flint admin panel, under Clients, reserve each rig's address so it never changes.
3. On each rig: `curl -I https://example.com`, and `ethtool enp0s31f6 | grep Speed` should say `1000Mb/s`.

RIG	CONNECTION	ADDRESS
Rig 2	Ethernet, Flint port 2	`192.168.8.213`
Rig 1	Ethernet, Flint port 3	`192.168.8.112`

Traffic between the rigs stays on the Flint and never touches the school network. Only internet traffic uses the Mac mini's Wi-Fi.

Measured afterwards (2026-10-22/23, link, rig to rig, internet): both rigs link at 1 Gb/s full duplex. `iperf3` between them carries 934 Mb/s each way, and the router answers in ~0.5 ms. The internet gets ~60–90 Mb/s each way, shared: ~84 Mb/s combined with both rigs downloading. Rig 1 first linked at only 100 Mb/s; a cable or port change on 2026-10-22 fixed it.

Planned: the tunnel to EC2

Not built yet. These are the steps as planned. They continue in Sprint 5.

Step 6: Test which port the tunnel can use

WireGuard's default port, UDP 51820, is very likely blocked by the district. UDP 443 often gets through. nginx on the relay uses TCP 443, so UDP 443 doesn't conflict.

1. AWS console → the relay instance → Security → its security group → Edit inbound rules. Add Custom UDP, port 443, source `0.0.0.0/0`.
2. On the relay: `sudo nc -ul 443`
3. On the Mac mini (it must be the machine on the school Wi-Fi):

```
EC2_IP=$(dig +short ai.opencodingsociety.com)
echo hello | nc -u -w2 $EC2_IP 443
```

If `hello` appears on the relay, UDP 443 gets out: continue with Step 7. If nothing appears, UDP is blocked: stop, and use the TCP 443 fallback in Pitfalls. Ctrl+C stops the listener.

Step 7: WireGuard server on the EC2 relay

Run each block on the relay and check for errors before the next. This adds WireGuard beside the existing nginx without changing it.

```
# 7a. Install WireGuard and turn on forwarding
sudo apt update && sudo apt install -y wireguard
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-wg.conf
sudo sysctl --system

# 7b. Server keys
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/server_private.key \
    | wg pubkey > /etc/wireguard/server_public.key'

# 7c. Server config
IFACE=$(ip route show default | awk '{print $5}')
SERVER_PRIV=$(sudo cat /etc/wireguard/server_private.key)
sudo tee /etc/wireguard/wg0.conf >/dev/null <<EOF
[Interface]
Address = 10.8.0.1/24
ListenPort = 443
PrivateKey = $SERVER_PRIV
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o $IFACE -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o $IFACE -j MASQUERADE
EOF
sudo chmod 600 /etc/wireguard/wg0.conf

# 7d. The Flint's keys, registered as a peer
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/flint_private.key \
    | wg pubkey > /etc/wireguard/flint_public.key'
sudo tee -a /etc/wireguard/wg0.conf >/dev/null <<EOF
[Peer]
PublicKey = $(sudo cat /etc/wireguard/flint_public.key)
AllowedIPs = 10.8.0.2/32
EOF

# 7e. Start it, and keep it across reboots
sudo systemctl enable --now wg-quick@wg0
sudo wg show                  # interface wg0, listening port 443
sudo ufw allow 443/udp        # only if ufw is enabled

# 7f. Print the Flint's config
EC2_IP=$(curl -s https://checkip.amazonaws.com)
cat <<EOF
[Interface]
PrivateKey = $(sudo cat /etc/wireguard/flint_private.key)
Address = 10.8.0.2/32
DNS = 1.1.1.1
MTU = 1380

[Peer]
PublicKey = $(sudo cat /etc/wireguard/server_public.key)
Endpoint = $EC2_IP:443
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
EOF
```

Save 7f's output on your laptop as `flint.conf`.

> **CAUTION**
>
> `flint.conf` contains a private key. Never paste it into GitHub, Slack, a shared doc, or a chat. If it leaks, delete the peer on EC2 and redo 7d and 7f.

SETTING	WHAT IT DOES
`ListenPort = 443`	The relay listens for WireGuard on UDP 443, the port that passed Step 6
`PostUp ... MASQUERADE`	Tunnel traffic leaves to the internet from EC2's public IP
`AllowedIPs = 0.0.0.0/0` (Flint)	All of the Flint's internet traffic goes into the tunnel (full tunnel)
`PersistentKeepalive = 25`	A small packet every 25 s keeps the school's NAT entry open
`MTU = 1380`	Leaves room for tunnel overhead so large packets aren't dropped

Step 8: WireGuard client on the Flint 2

Menu names vary a little between firmware versions.

1. Open `http://192.168.8.1` from a device on the OCS LAN.
2. VPN → WireGuard Client → Add a New WireGuard Configuration → upload (or paste) `flint.conf`. Name it `ocs-ec2` and save.
3. Turn the toggle on to connect.
4. In the VPN options, choose **Global Proxy** so every device on the OCS LAN uses the tunnel.
5. Turn on **Block Non-VPN Traffic** (the kill switch), so nothing leaks out through the school filter if the tunnel drops.

> **NOTE**
>
> With the kill switch on, the whole OCS LAN loses internet when the tunnel is down; traffic between local devices keeps working. That's intended. Check Step 9 first whenever the internet stops. Other groups share the LAN (ports 5 and 6), so agree on this with them before turning it on.

Step 9: Test the tunnel end to end

#	WHERE	COMMAND OR ACTION	PASS LOOKS LIKE
1	EC2 relay	`sudo wg show`	Peer `10.8.0.2` has a recent handshake, and the transfer counters climb
2	Rig 2	`curl -s https://ifconfig.me`	The EC2 Elastic IP, not the school's address
3	Rig 1	`curl -s https://checkip.amazonaws.com`	The same EC2 IP
4	Any device	Open a site the district normally filters	It loads

#	WHERE	COMMAND OR ACTION	PASS LOOKS LIKE
5	Flint admin	Turn the VPN off for a moment	Clients lose internet (the kill switch works). Turn it back on
6	Mac mini	`sudo reboot`, wait, then repeat 1–3	Everything comes back with no manual steps

Step 10: Reach the rigs from EC2 through the tunnel

After Step 9 passes:

1. On EC2, change the Flint peer's line in `/etc/wireguard/wg0.conf` to `AllowedIPs = 10.8.0.2/32, 192.168.8.0/24`, then `sudo systemctl restart wg-quick@wg0`.
2. On the Flint, in the WireGuard client settings, turn on remote access to the LAN.
3. From EC2, ping or curl each rig at its reserved address, for example `curl http://192.168.8.112:<port>/`. Both rigs should answer.

How to check it

CHECK	PASS LOOKS LIKE	IF IT FAILS
Mac mini: `ip route`	Default route on `dev wlp2s0`	`nmcli connection up "PUSD open test"`; see Pitfalls
Mac mini: `nmcli device`	`wlp2s0` connected, `enp3s0f0` connected (`brume-link`)	Re-run Step 1's checks
Mac mini: `sysctl net.ipv4.ip_forward`	`1`	`nmcli connection up brume-link`
Flint admin → Internet	Ethernet, DHCP, a `10.42.0.x` address	The cable is in a LAN port instead of WAN, or `brume-link` is down
Each rig: `ethtool enp0s31f6 | grep -E 'Speed|Duplex'`	`1000Mb/s`, `Full`	Swap the cable or the Flint port (Rig 1 linked at 100 Mb/s until a cable/port change)
Each rig: `curl -I https://example.com`	HTTP headers	Check the chain upstream: the Flint, then the Mac mini
After Step 9: each rig `curl -s https://ifconfig.me`	EC2's Elastic IP	`sudo wg show` on EC2

Pitfalls

SYMPTOM	LIKELY CAUSE	FIX
`wlp2s0` missing from `nmcli device`	`wl` driver not loaded (kernel update or Secure Boot)	Re-run Step 1's checks; reinstall `broadcom-sta-dkms`; enroll the MOK if Secure Boot is on. Use the USB dongle meanwhile
`PUSD open test` won't connect	The pinned access point is down, or the Mac mini moved rooms	Redo Step 2's scan and update the BSSID with `nmcli connection modify`

SYMPTOM	LIKELY CAUSE	FIX
Router has no WAN address	Cable in a LAN port, or `brume-link` is down	Use the WAN port; `nmcli connection up brume-link`
`ping` fails but browsing works	The school drops ICMP	Normal. Test with `curl`
A rig links at 100 Mb/s	Cable or port	Swap the cable or move to another Flint port, then re-check `ethtool`
No handshake in `wg show`	UDP 443 blocked, security group rule missing, or wrong Endpoint	Re-check Step 6 and the inbound rule; the Endpoint must be the Elastic IP
Handshake works but some sites hang	MTU too high	Set `MTU = 1280` in `flint.conf` and re-upload it
UDP blocked entirely	The district filters all outbound UDP	Fall back to OpenVPN over TCP 443 (the Flint supports it). nginx already owns TCP 443 on the relay, so it needs port sharing or a second address
The whole LAN lost internet	Tunnel down, with the kill switch on	`sudo wg show` on EC2, and the VPN page on the Flint

Fallback uplink: the Mac mini's USB dongle can carry the school connection on the encrypted OWE side: `nmcli connection up "PUSD onboard" ifname $(nmcli -t -f DEVICE device | grep ^wlx)`.

Open items

- **The tunnel (Steps 6–10)** isn't built. Sprint 5, networking.
- **Mac mini reboot test** with the USB dongle unplugged: pending. It must pass before anything depends on the uplink.
- **The uplink is unencrypted.** The built-in card is on the open side of the school Wi-Fi. Until the tunnel is live, anything not using HTTPS can be seen by others on that Wi-Fi.
- **Throughput.** Every byte to the internet crosses one Wi-Fi link and the Mac mini (and, once built, EC2). Measured ~84 Mb/s shared on 2026-10-23. The hops haven't been measured one by one.
- **Double NAT.** The Mac mini and the Flint both NAT. Outbound is fine. Inbound access should come through the tunnel, not port forwarding.
- **Single points of failure:** one Mac mini, one pinned access point, one out-of-tree driver, and one EC2 instance carry the whole network. `wl` is unmaintained, so a routine kernel update can take down the uplink.
- **The LAN is shared.** Other groups use Flint ports 5 and 6, so their devices are on the rigs' subnet. The rigs' orchestrators listen on `0.0.0.0:9000` with no host firewall (status). Firewalling them is a Sprint 5 item.
- **EC2 becomes the trust boundary** for both rigs once the tunnel is up. Keep its security group tight and its keys private.

Sources

- The team's original PDF of this playbook (2026-10-16), replaced by this version on 2026-10-23. It's in the git history at `evidence/sprint4/OCS_Rig_Routing_Playbook.pdf`

- Ports, SSID, and 5 GHz-only Wi-Fi: reported by the team on 2026-10-23
- Measurements: Rig 1 gigabit link, rig-to-rig iperf3, internet throughput
- Network, the canonical description of the LAN